

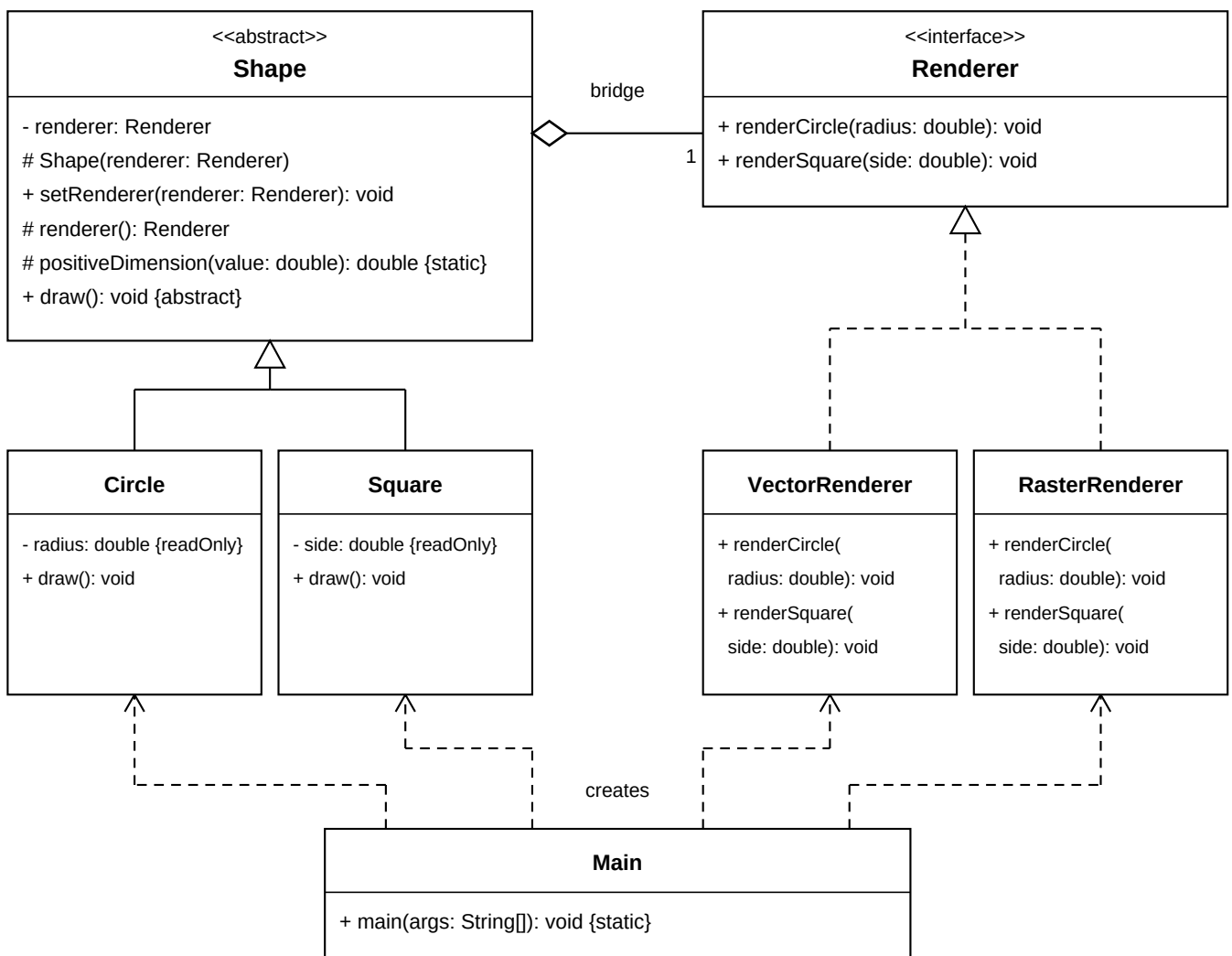
Assignment 3 Bridge Pattern

Introduction

The chosen topic is Shape-Renderer. Circle and Square represent shapes, while VectorRenderer and RasterRenderer represent ways to draw them. Bridge fits because a shape stores a Renderer reference and delegates drawing to it. The shape classes do not depend on a specific renderer class.

Main shares one vector renderer and one raster renderer between both shapes. It draws with vector rendering, switches each shape to raster rendering, then switches the same circle back to vector rendering. Console output describes each operation. Bridge separates two design dimensions; Adapter makes an existing incompatible interface usable.

UML class diagram



Hollow triangles: inheritance / interface realization. Open arrows: client dependencies.

Hollow diamond: shared reference (object composition in code, no exclusive ownership).

Main is in bridge; Shape, Circle, and Square are in bridge.shape; Renderer and its implementations are in bridge.renderer. Main creates the concrete objects and works through Shape and Renderer references. The hollow diamond shows the shared renderer reference. Concrete constructors and final method modifiers are omitted for readability.

Clean Code principles

The following excerpts are from the implementation. Each explanation states the design choice and its benefit.

1 Single responsibility

```
renderer().renderCircle(radius);
```

Circle keeps its radius and delegates drawing. VectorRenderer and RasterRenderer decide how to describe their drawing operations, so rendering details stay out of the shape classes.

2 Depend on abstractions

```
private Renderer renderer;
```

Shape refers to the Renderer interface rather than a concrete renderer. Both implementations can be supplied and switched without changing the shape classes.

3 Fail fast

```
Objects.requireNonNull(renderer, "Renderer must not be null");
```

Shape rejects null renderers during construction and switching. Its positiveDimension helper rejects zero, negative, NaN, and infinite dimensions before an invalid shape is created.

4 Avoid duplicated logic

```
this.radius = positiveDimension(radius);  
this.side = positiveDimension(side);
```

These constructor excerpts are from Circle and Square. Both reuse the validation helper in Shape, so the dimension rule is defined in one place.

5 Open closed principle

```
Shape circle = new Circle(5.0, vector);  
circle.setRenderer(raster);
```

Main chooses implementations through the Renderer interface. A new renderer can implement both drawing operations without editing Shape, Circle, or Square.

Conclusion

Bridge avoids separate classes for every shape-renderer combination and allows runtime switching. New renderers do not require changes to the shape classes. The trade-off is extra classes and delegation. This interface explicitly supports circles and squares, so a new shape kind requiring another drawing operation would also require changes to Renderer and its implementations. Rendering is simulated through console output.

GitHub repository

<https://github.com/asankhanshyngys/assignment-3-bridge-pattern>